CSCE 434, Spring 2026 **Lab 3: Instruction Scheduling**

Table of Contents		
1.	Introduction	1
2.	Overview of the Problem	2
3.	Code Specifications	3
4.	Submitting Your Code	5
5.	Grading Rubric	6
6.	Honor Policy	7
A.	ILOC Virtual Machine	8
B.	ILOC Subset & Simulator	9
C.	Tools	11
D.	Checklist for Lab 3	13

Critical Date for the Project	
Code Due Date	04/24/2025

Please report suspected typographical errors to the instructor via email.

1. Introduction

In this project you will design, test, and implement an instruction scheduler that operates on a single basic block of ILOC code. The scheduler should rearrange the instructions in the input block to reduce the number of cycles required to execute the output block, as measured with the Lab 3 ILOC simulator. (See § A for details related to the Lab 3 ILOC simulator.)

Your scheduler will take as input a file that contains a sequence of operations, expressed in the ILOC subset used in the two prior projects (see A). As output, your scheduler will produce an "equivalent" sequence of ILOC operations, reordered to execute in fewer cycles and re-written in a notation to encode two operations executing in the same cycle.

> *For the purposes of this lab, an input program and an output program are considered* ***equivalent*** *if and only if they print the same values in the same order to* ***stdout*** *and each data-memory location defined by an execution of the input program receives the same value when the output program executes.*

The input blocks receive input in one of two ways. Either the data is hard-coded into the block, or you provide that input on the command line using the simulator's **-i** command-line option.

You have wide latitude in your choice of scheduling algorithms, ranging from classic list scheduling through a novel technique of your own invention. See the project videos and Chapter 12 of *Engineering a Compiler* (including the chapter notes) for descriptions of scheduling algorithms to consider. You should watch the videos and read this document completely before you begin to design and code your scheduler.

One of the central data structures in an instruction scheduler is a dependence graph. Many students have found it useful to look at dependence graphs using a visualization tool. We strongly recommend that your scheduler include an option to build an output file that can be visualized using Graphviz (§ C.3) or some equivalent application.

2. Overview of the Problem

An *instruction scheduler* is a compiler pass that reorders the operations in the code to reduce the number of cycles that the code takes to execute. It does not change the operations (other than register renaming). It does not eliminate operations. It simply reorders the operations.

The scheduled code must produce the same results as the original code, without relying on the hardware interlocks provided in the ILOC Simulator. A hardware interlock, when available, occurs when the hardware tries to issue an operation *s* followed by an operation *t.* If *t* uses a register that *s* defines, and if *s* has not yet retired (completed its operation), then the interlock will delay the start of *t* until *s* writes its results (the results become available for *t*).

Many processors support hardware interlocks. On a processor without interlocks, ***the compiler must ensure*** that no operation issues before its operands are ready. If the compiler cannot fill the slots between *s* and *t* with useful operations from the code, it must insert **nop** operations to forcibly delay the start of *t.*

In this project, you will schedule blocks of ILOC code with the assumption that the ILOC simulator that ***does not*** enforce interlocks. To ensure that operations only issue after their operands are ready, your scheduler may need to insert **nop**s to delay operations.

The Lab 3 ILOC simulator supports several interlock modes, controlled by the **-s** command line flag. It is important that you use the correct interlock modes at different points in your testing and debugging (see § A). Failure to do so may lead you to believe that your scheduled code is correct when it is not. The testing scripts automate selection of the interlock modes.

The Lab 3 ILOC Simulator (see § A) has a couple of other complications that will make scheduling a challenge:

- The **load**, **store**, and **mult** operations all have multi-cycle latencies (Table B on page 11). Operations read their operands in the cycle when they issue. They write their results in the cycle when they retire. If an operation *t* tries to use the result of an operation that has not retired, it may access an incorrect value. To ensure correct execution, the scheduler must ensure that operations only issue when their operands are available.
- The Lab3 simulator has ***two functional units***. Each unit can execute a subset of the ILOC operations. If no operation is available to execute on one of the units, then that unit sits idle until the next cycle. The scheduler should arrange, whenever possible, to have an operation issue for <u>each of the functional units</u> in <u>each cycle</u>. The scheduler has a new notation in ILOC to indicate which operations should issue together. It encloses the pair of operations in square brackets and separates them with a semicolon, as in

  [load r4 => r5; add r1, r2, => r3]

  The key to a fast schedule is making full use of both functional units and reducing the number of **nop**s that the scheduler must insert.

  Note that this two-operation notation is the ***output*** of your scheduler, not the input to it. The input is still in the form of the ILOC programs from Lab 1 and Lab 2. In other words, you will ***not*** have to change your scanner and parser

3. Code Specifications

Your scheduler must adhere to the following specifications.

- **Name:** The executable version of your instruction scheduler must be named **schedule.**
- **Behavior:** Your scheduler must work in two distinct modes, as controlled by parameters and flags on the command line.

schedule –h	When a –h flag is detected, **schedule** must produce a list of valid command-line arguments that includes a description of all command-line arguments required for Lab 3 as well as any additional command-line arguments supported by your schedule implementation. **schedule** is not required to process command-line arguments that appear after the –h flag.
schedule <name>	This command will be used to invoke **schedule** on the input block contained in <name>. <name> specifies the name of the input file. <name> is a valid Linux pathname relative to the current working directory. **schedule** will produce, as output, an ILOC program that is *equivalent* to the input program, albeit reordered to improve (*shorten*) its execution time on the Lab 3 ILOC Simulator. The output code must use the square bracket notation [op_1 ; op_2] to designate operations that should issue in the same cycle. **schedule** is not required to process command-line arguments that appear after <file name>.

Your scheduler must check the correctness of the command-line arguments and produce reasonable and informative error messages when errors are detected. The output block must be printed to the standard output stream (stdout); error messages must be printed to the standard error stream (stderr).

The input language for Lab 3 is the same as in previous projects. You should reuse your ILOC front end from Lab 1 and your renaming pass from Lab 2. If you elect to rewrite the front end, it still must process the input one character at a time.

If your scheduler cannot read the specified file, or the code in the file is not valid ILOC, it should produce a reasonable and informative message. If the ILOC code in the input file uses a value from a register that has no prior definition, your instruction scheduler should handle this situation gracefully.

- **Restrictions on Optimizations: Schedule** must <u>not</u> perform optimizations other than instruction scheduling and register renaming. Examples of optimizations that will result in loss of credit include:
  - *Removing Dead Code:* **Schedule** may remove **nop**s found in the input file; they have no effect on the equivalence of the input and output programs. Any other operation found in the input code must appear in the final code.
  - *Adding ILOC Operations:* **Schedule** may add **nop**s. It may not add other ILOC operations. For example, **schedule** may not add **loadI** operations.
  - *Folding Constants:* **Schedule** may use constant propagation to disambiguate memory references. You may not use constant propagation to eliminate computations.
  - *Optimizing Based on the SIM INPUT Directive:* **Schedule** may not rely on input constants from the block's *SIM INPUT* directive. **Schedule** is expected to generate correct code for any set of input values.
- **Makefile & Shell Script:** As in Labs 1 and 2, you will submit a tar archive file that contains the source code for your allocator, along with any scripts or **Makefile**s required to create and execute the final code.

  Lab 3 submissions written in languages that require a compilation step, such as C, C++, or Java, must include a **Makefile**.[1] For a Java submission, you must submit the code, not a **jar** file. The autograder will invoke **make** with the two commands:

  make clean
  make build

  in that order. The **clean** target should remove any files from old attempts to build the executable. The **build** target should ensure that the **schedule** executable is ready to run.

  If your submission is written in a language that does not require compilation, such as python, then it does not need a **Makefile**.

  If your submission does not create a standalone executable named **schedule**, then it should include an executable shell script named **schedule** that accepts the required command-line arguments and passes them to the program. For example, a project written in python[2] named lab3.py could provide an executable shell script named **schedule** that includes the following instructions:

  #!/bin/bash
  python lab3.py $@

  To invoke python3, you would use the command "python3" rather than "python".

  Similarly, a project written in Java with a jar file named lab3.jar or a class named

[1] If you prefer to use a build manager other than **make**, you may invoke that build manager from your **Makefile**. Your build manager should not leave behind files — particularly hidden files in CSCE-434 directory on **linux2**.

[2] Alternatively, you can write a python program that runs directly. As an example, see the lab 1 testing script on LINUX2. Its first line is a comment that tells the shell to invoke python; the main routine parses argv and argc.

lab3.class that contains the main function could provide, respectively, one of the following two executable shell scripts, naming it **schedule**.

```
#!/bin/bash
java  -jar lab3.jar  $@
```

```
#!/bin/bash
java  lab3  $@
```

To ensure that your **schedule** shell script is executable on a Linux system, execute the following command in the LINUX2 directory where your **schedule** shell script resides:

```
chmod a+x schedule
```

To avoid problems related to the translation of carriage return and line feed between Windows and Linux, we strongly recommend that you write your shell script and **Makefile** on LINUX2 rather than on a Windows system (see also **man dos2unix**).

- **README:** Your submission must include a **README** file (name in uppercase letters) that provides directions for building and invoking your program. Include a description of all command-line arguments required for Lab 3 as well as any additional command-line arguments that your scheduler supports. The first two lines of your README file should be in the following format:

  ```
  //NAME: <your name>
  //NETID: <your NetID>
  ```

  Note that there are no spaces after the slashes, and that the keywords **NAME** and **NETID** are in all capital letters. Your name should be capitalized normally. Your netid should be lowercase characters and numbers.

- **Programming Language:** You may use any programming language available on A&M's LINUX2 facility, except for Perl. Your goal should be to use a language that is available on LINUX2, in which you are comfortable programming, for which you have decent debugging tools, and that allows you to easily reuse code from Lab 2.

- **USE LINUX2:** Your submission must work on A&M's LINUX2 systems. If the code must be translated (e.g., compiled, linked, turned into a jar or an a.out), that must happen on LINUX2. Test the code on LINUX2; test it using the Python testing script found in the Lab3 folder. Students may find differences between language and library versions on their laptops and on LINUX2. You want to find those problems early and resolve them.

4. Submitting Your Code

Due date: The due date for your code submission is shown in the table on page 1. All work is due at 11:59PM on the specified day. Individual extensions to this deadline will not be granted.

Early-Submission Bonus: Submissions received before the due date will be awarded an additional two points per day up to a maximum of eight points.

Late Penalty: Submissions received after the due date will lose two points per day. Late code will be automatically accepted for seven days after the due date. To submit code after that deadline, you must obtain express permission from the instructors. Contact the instructor via email to request permission.

Late Penalty Waivers: To cover situations that inevitably arise, we will waive up to six days of late penalties per semester when computing your final CSCE 434 grade. Note that we choose where those days apply, after all grades are complete. We will apply these "grace" days in a way that maximizes benefit to you.

Submission Details: To submit your work, you must create a tar archive that contains your submission, including (1) the source code for the register allocator, (2) the **Makefile** and/or shell script, (3) the **README** file, and (4) any other files that are needed for the autograder to build and test your code. The tar file should unpack into the current working directory. The **README** and **Makefile** or executable script must reside in that top-level directory.

The document "**NoteOnTarFiles.pdf**", available on both the CSCE 434 Canvas Site and in the **/mnt/class_files/Classes/CSCE-434/Class/lab1** directory on LINUX2 provides more details about the structure of the submission file system and the tar archive. You can test your archive using the scaled-down grader. If your archive does not work with the grader, that will reduce the points you receive in the conforms-to-specs part of the grading rubric.

If your scheduler does not work, include in your tar file a file named STATUS that contains a brief description of its current state. You may include ILOC files that your lab handles correctly and ILOC files that it handles incorrectly. If you include such files, describe them in both the README and the STATUS files.

Name the tar file with your A&M netid (e.g., a person with netid jed12 would create an archive named jed12.tar). Submit your tar file through Gradescope. You should keep the tar archive until the end of the semester, to record of what you submitted.

5. Grading Rubric

The Lab 3 grade accounts for 20% of your final CSCE 434 grade. The code rubric is based on 200 points. Correctness and cycle counts will be determined using the Lab 3 ILOC simulator.

- **80 points** for the correctness of the code produced by **schedule** across the collection of blocks tested in the autograder. [The "stripped down autograder" does not include all the test blocks that we will use in our version of the autograder.]
- **60 points** for effectiveness, measured as the number of cycles required to run the scheduled code produced by your submission on the Lab 3 ILOC simulator. A scheduler that gets within 10% of the cycle counts for lab3_ref will receive full credit.
- **60 points** for scalability and efficiency. These points will be awarded for two different criteria: efficiency and scalability. The autograder will time your allocator on the **SLOCs** and **RQ** timing blocks, found at **/mnt/class_files/Classes/CSCE-434/Class/ILOC/Scalability/**. Points toward your grade will be based on the measured behavior of your submission.

  Scalability: The front end should display linear scaling — that is, a doubling of the input size should produce growth of no more than 2x in the runtime. You can see this by plotting runtime as a function of non-comment lines in the input file; do not use a logarithmic scale on either axis.

Efficiency: The table below shows the measured runtime on block T128k.i required for full credit and the time that will produce zero credit. Runtimes between full credit and zero credit will receive partial credit, on a linear scale between the breakpoints. The specific breakpoints were determined by analyzing the lab 3 submissions from the Fall 2020 class.

Language	Full Credit	No Credit
C	≤ 1.5 second	≥ 3.0 seconds
C++	≤ 2.0 second	≥ 4.0 seconds
Java	≤ 2.75 second	≥ 10.0 seconds
Python	≤ 10 seconds	≥ 20.0 seconds
Go	≤ 1.5 second	≥ 3.0 seconds

As before, the instructor will determine a set of breakpoints for languages not listed in the table, based on the language, its implementation, and, perhaps, some additional testing.

By this point in CSCE 434, we expect that you understand how to submit a code distribution that works. Thus, we are not awarding points for "conforms to specifications." Nonetheless, the grader will test the code. Any submission that fails to conform to specification or requires manual intervention to run will **lose 20 points**

6. Honor Policy

As in Labs 1 and 2, your submitted scheduler and **README** file must consist of code and/or text that you wrote, not edited or copied versions of code and/or text written by others or in collaboration with others. You may not look at CSCE 434 code from your classmates or from past semesters. Your submitted code may not invoke the CSCE 434 reference scheduler, or any other scheduler that you did not write.

You are free to collaborate with current CSCE 434 students when preparing your **Makefile** and/or shell script and to submit the results of your collaborative **Makefile** and/or shell script efforts. However, as indicated in the previous paragraph, all other Lab 3 code and text submitted must be your own work, not the result of a collaborative effort.

You are welcome to discuss Lab 3 with the CSCE 434 staff and with students currently taking CSCE 434. You are also encouraged to use the archive of test blocks produced by students in previous semesters. However, we ask that you not make your CSCE 434 labs available to students (other than CSCE 434 TAs) in any form during or after this semester. We ask that you not place your code anywhere on the Internet that is viewable by others.

Appendix A ILOC Virtual Machine

The ILOC Virtual Machine has disjoint address spaces for instructions and data. The Lab 3 simulator provides 200,000 registers, numbered r0 through r199999, and four megabytes of available data memory; accesses to memory must be word aligned. By convention, ILOC input blocks reserve memory addresses above 32,767 for Lab 2 spill code.

ILOC Microarchitecture. To make the task of scheduling more interesting, the target machine's behavior is more complicated than it was in Lab 2. In Lab 3, the ILOC virtual machine has two functional units, f_0 and f_1. These two units are identical, except

- Only f_0 can execute the load and store operations. Either f_0 or f_1 can execute loadI.
- Only f_1 can execute the mult operation.
- Only one output operation can execute per cycle. It can execute on either f_0 or f_1.

Thus, most instructions can execute on either functional unit. Only the long latency instructions are constrained to specific functional units. The restriction on output is needed to ensure determinism in the print stream, which is necessary given that your scheduler is required to preserve the ordering among the output instructions.

The ILOC machine expects parallel operations to use the following syntax. [op_1 ; op_2] is a single instruction that contains two ILOC operations, op_1 and op_2. op_1 and op_2 issue concurrently. The order of the operations within the pair is not important provided the machine constraints are not violated. Your scheduler must emit its output in this multi-operation format.

The multicycle operations, **load**, **store**, and **mult**, read their operands in the cycle when they issue. They write their results in the cycle when they retire. These three operations are *non-blocking*—that is, the simulator continues to issue successive instructions while the multicycle operations execute, unless one of those later operations tries to access a value defined by a multicycle operation that has issued but not yet retired. At that point, the simulator's behavior depends on the ***interlock setting***.

- With register interlocks enabled (the **-s 3** flag), the simulator will stall an operation when it tries to access a register value that will be defined by an operation that has issued but not yet retired. The simulator will stall the functional unit that holds the operation until the defining operation retires.
- With register interlocks disabled (the **-s 1** flag), the operation will receive the value that is in the operand register at the start of the cycle when the second operation issues. That value is unlikely to be the intended value, so this condition typically produces incorrect or unpredictable behavior.

Thus, when testing scheduled code, you should use the **-s 1** setting on the simulator. When running the unscheduled code to test for correctness, use the default setting, **-s 3**. If you use the testing script, it manages these settings for you.

Appendix B. ILOC Subset & Simulator

ILOC Subset: Lab 3 input and output files consist of a single block of code written in a subset of ILOC. Your scheduler must support and restrict itself to the following case-sensitive operations:

Syntax					Meaning	Latency
load	r1		=>	r2	r2 ← MEM(r1)	5
loadI	x		=>	r2	r2 ← x	1
store	r1		=>	r2	MEM(r2) ← r1	5
add	r1,	r2	=>	r3	r3 ← r1 + r2	1
sub	r1,	r2	=>	r3	r3 ← r1 - r2	1
mult	r1,	r2	=>	r3	r3 ← r1 * r2	3
lshift	r1,	r2	=>	r3	r3 ← r1 << r2	1
rshift	r1,	r2	=>	r3	r3 ← r1 >> r2	1
output	x				prints MEM(x) to stdout	1
nop					idle for one cycle	1

All register names have an initial lowercase r followed immediately by a non-negative integer. Leading zeros in the register name are not significant; thus, r017 and r17 refer to the same register. Arguments that do not begin with r, which appear as x in the table above, are assumed to be non-negative integer constants in the range 0 to $2^{31} - 1$. The assignment arrow is composed of an equal sign followed by a greater than symbol, as shown (=>).

Each ILOC operation in an input block must begin on a new line and be completely contained on that line.[3] Blanks and tabs are treated as whitespace. The opcode must be followed by whitespace. Whitespace preceding and following all other symbols is optional. Whitespace is not allowed within operation names, register names, or the assignment arrow. A double slash (//) indicates that the rest of the line is a comment and can be discarded. Empty lines and nops in input files may also be discarded.

ILOC Simulator: Lab 3 uses a different configuration of the ILOC simulator than does Lab 2. As expected, the simulator executable, source and documentation are available at **/mnt/class_files/Classes/CSCE-434/Class/lab3** on LINUX2.

The Lab 3 simulator builds and executes on LINUX2. You can either run the simulator from the above directory or copy it into your local directory. The simulator appears to work on other OS implementations but is not guaranteed to work on other OS implementations. Your instruction scheduler will be tested and graded on LINUX2, so you should test it on LINUX2.

Simulator Usage Example: If an input file named test1.i is in your present working directory,

[3] Carriage returns (CR, \r, 0x0D) and line feeds (LF, \n, 0x0A) may appear as valid characters in end-of-line sequences.

you can invoke the simulator on CLEAR on test1.i in the following manner:

/mnt/class_files/Classes/CSCE-434/Class/lab3/sim -s 3 -i 2048 1 2 3 < test1.i

This command will cause the simulator to execute the instructions in test1.i, print the values corresponding to ILOC output instructions, and display the total number of cycles, operations, and instructions executed.

The -s NUM parameter is optional. It controls the handling of stalls that simulate hardware interlocks. By default, the Lab 3 ILOC simulator uses -s 3, which enforces interlocks on branches, registers, and memory locations. When you run the original code for an ILOC test block to determine the correct output results, you should use the default (-s 3 option). When you test the code produced by your Lab 3 scheduler, you should use the -s 1 option, which only enforces interlocks on branches. The output of your scheduler will be graded with the -s 1 option on the simulator; you should always use -s 1 when you run code that is output from your scheduler or from lab3_ref.

The -i parameter is used to fill memory, starting at the memory location indicated by the first argument that appears after -i, with the initial values listed after the memory location. The Lab 3 ILOC simulator has byte addressable memory, but the ILOC subset for Lab 3 only allows word-aligned accesses. So, in the above example, 1 will be written to memory location 2048, 2 to location 2052, and 3 to location 2056. (This means that, before the memory locations are overwritten during program execution, "output 2048" will cause 1 to be printed by the simulator, "output 2052" will cause 2 to be printed, etc.)

See the ILOC simulator document for additional information about supported command-line options. (Command-line options -d, -r, -x, and -c are not relevant for Lab 3.)

ILOC Input Blocks: A large collection of ILOC input blocks is available on LINUX2, in the directory **/mnt/class_files/Classes/CSCE-434/Class/ILOC/**. Each block has the //SIM INPUT: and //OUTPUT: specifications to allow the various testing scripts to check for correctness. If you experience problems with the input blocks, please submit bug reports to the instructor.

Appendix C. Tools

C.1 Reference Instruction Scheduler

To help you understand the functioning of an instruction scheduler for a single basic block and to provide an exemplar for your implementation and debugging efforts, we provide the CSCE 434 reference instruction scheduler. The reference instruction scheduler is a C implementation of an instruction scheduler for a single basic block. You can improve your understanding of instruction scheduling by examining its output on small blocks. You will also use the reference instruction scheduler to determine how well your instruction scheduler performs in terms of effectiveness (quality of the schedules that your instruction scheduler generates) and efficiency (runtime of your instruction scheduler).

The CSCE 434 reference instruction scheduler can be invoked on LINUX2 as follows:

/mnt/class_files/Classes/CSCE-434/Class/lab3/lab3_ref <name>

where <name> both specifies the name of the input file and is a valid Linux pathname relative to the current working directory. For a description of the flags supported by the CSCE 434 reference instruction scheduler, specify the -h flag on the command line.

As with Lab 2, a testing script is available in the lab3 directory on LINUX2.

Note that lab3_ref can only be run on LINUX2 and that it does not produce scheduled code for test blocks that use undefined registers

C.2 Testing and Timing Scripts

As in Labs 1 and 2, you will have access to scripts to test and time your implementation. The test script will assess your scheduler's correctness and effectiveness using a subset of the blocks that the production autograder will use. It reports raw scores for correctness and effectiveness.

The timer script will measure the efficiency of your scheduler and estimate its scalability.

> *It is somewhat tricky to measure the elapsed running time of an application on a busy system, so you should be aware that this process is noisy. If you see results that seem inconsistent, run the timer again at another time on another system.*

If the load factor for a system is high (e.g., above 2), you should try at another time on another system. Try COMPUTE.

C.3 Graphviz

The Graphviz tool is available for various forms of Unix, for Windows, and for the Mac OS from:

http://www.graphviz.org

It is not currently installed on any of the A&M-supported platforms. You can download a copy for your own machine. If this poses a problem, please see the teaching assistants.

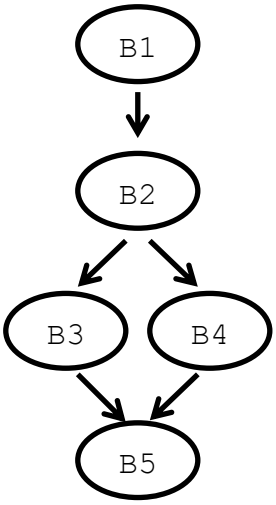
The point of using Graphviz on small input blocks <u>*early in your lab*</u> is to let you see the dependence graph and understand it in a way that is hard to achieve otherwise. As secondary benefit, you will gain experience using the kinds of tools that compiler writers use in their work.

Graphviz takes as input a *dot* file—a text file that represents, for our purposes, the nodes and edges in a directed graph. Nodes are named with arbitrary integers; edges are named with a pair of numbers joined by an "->" operator that specifies the direction of the edge. The full specification of the *dot* language can be found on the Graphviz web site. For this lab, you will work with a small subset of the *dot* language.

Here is a simple *dot* file and its output (redrawn in Microsoft Word):

```
digraph testcase1
{
 1 [label = "B1"];
 2 [label = "B2"];
 3 [label = "B3"];
 4 [label = "B4"];
 5 [label = "B5"];
 1 -> 2;
 2 -> 3;
 2 -> 4;
 3 -> 5;
 4 -> 5;
}
```

Input *dot* File

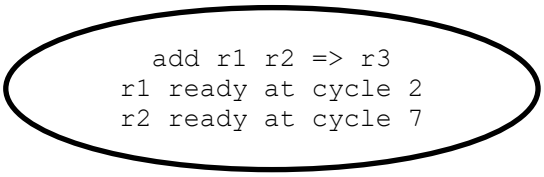


Output Drawing

Note that the "label" field can be arbitrarily long and can include newlines, so that a *dot* file with the following node specification:

```
1 [label="add r1 r2 => r3\nr1 ready at cycle 2\nr2 ready at cycle 7\n"];
```

should produce a node that looks like:



Using this feature of Graphviz, you can display as much information as you need to understand the graphs that your lab is building. Additional annotations on the nodes can greatly simplify your debugging effort.

Note: The use of Graphviz is optional, but strongly recommended for debugging small input blocks. Avoid using Graphviz for large input blocks because it takes significant computational time to produce graphs for large input blocks and, due to the large numbers of nodes involved, the large graphs are difficult to display and decipher.

Appendix D. Checklist for Lab 3

The following high-level checklist is provided to help you track progress on your Lab 3 code, report, and test block.

- ☐ Implement an instruction scheduler for a single basic block using a scheduling algorithm that you choose or devise.
- ☐ Use the Lab 3 ILOC simulator to ensure that the scheduled code produced by your program is correct and to measure the number of cycles that the ILOC simulator requires to execute each scheduled block. A variety of ILOC input blocks are available for you to use when testing your instruction scheduler. Use the -s 1 option for the simulator when testing your output code.
- ☐ Test your instruction scheduler on LINUX2. It will be graded on LINUX2. Measure its scalability and efficiency, and tune its performance if necessary.
- ☐ Create a new directory containing your submission. Construct a tar archive file from it.
- ☐ Submit your tar archive using Gradescope. Keep the submitted tar archive until next semester.

Revision History
Version 1.0: March 12, 2025
— *correct paths to scripts and executables on LINUX2*